

BCPNN–LTSL: a Bayesian Confidence Propagation Network on a Logarithmic Temporal Scale Ladder for Multivariate Time-Series Forecasting and Value-at-Risk Estimation

Design, Implementation, Codebase Development, Training, and Evaluation

Florian Fiebig

BCPNN simulator library by Anders Lansner

September 2026

Abstract

We describe a brain-inspired forecasting stack that couples a Bayesian Confidence Propagation Neural Network (BCPNN) to an explicit logarithmic temporal scale ladder (LTSL) of time-causal, time-recursive smoothing kernels in the sense of Lindeberg's temporal scale-space theory. The LTSL maintains K geometrically spaced leaky integrators over a radial-basis-function encoding of the input, giving the network a compact multi-scale memory of the past that requires no buffering, never accesses the future, and thus enables continual learning. The BCPNN associates that multi-scale code with the next observation and importantly, at every step, emits a full discrete predictive distribution rather than a point estimate. We trace the system from a Mackey–Glass proof of concept through S&P 100 equities to daily foreign-exchange rates, document the switch from a (trailing mean, first derivative) feature set to a (log return, trailing variance) feature set, and describe the continual, walk-forward *multislice* training schedule that lets a single network learn across instruments and calendar blocks without resetting weights. Three implementation defects that biased the predictive tail are identified and corrected, and their repair is verified against a synthetic Gaussian probe with a known 1 % quantile. On the four FX pairs used by the DeepVaR study (Fatouros et al., 2023) and over the identical 2018-01-01 to 2020-12-18 backtest window, BCPNN–LTSL passes all twelve Kupiec, Christoffersen independence and conditional-coverage tests, whereas DeepVaR is rejected on two; our 99 % VaR is 44 % less capital-intensive at equal-or-better tick loss. The point forecast, by contrast, carries no daily directional information. We argue that this asymmetry — predictable magnitude, unpredictable sign — is the correct reading of the data and the reason a distributional model is the right tool for this problem. Our direct comparison with DeepVaR demonstrates that BCPNN–LTSL enables a very fast and compute efficient feed-forward prediction of Value-At-Risk without sacrificing performance against the state-of-the-art, and thus improves on the latter.

1 Introduction

Most neural forecasters for financial time series are trained to minimise a point-forecast loss and then, if a risk number is required, wrap a distributional assumption around the residuals. We believe the struggles of these models reveals that this ordering is backwards. The sign of tomorrow's return is very close to a coin flip, while its magnitude is strongly autocorrelated — i.e. volatility clusters predictably, even if tomorrows value is almost impossible to predict. A model whose training signal is dominated by the direction of the next move therefore spends its capacity on the one component of the signal that is essentially unpredictable.

Our approach here takes the opposite route. The network is a **Bayesian Confidence Propagation Neural Network** (BCPNN), a cortically inspired associative architecture originally derived from Bayes' rule by Lansner and Ekeberg. Its units are organised into hypercolumns of mutually normalising minicolumns, so the activity of a population *is* a probability distribution over the feature values that population encodes. A BCPNN therefore produces a full predictive probability mass function (pmf) as its native output; a point forecast and a Value-at-Risk (VaR) quantile are both simple forward read-outs of the same object, obtained at no extra cost and with no distributional assumption bolted on afterwards.

Temporal context is supplied by an explicit **logarithmic temporal scale ladder** (LTSL). Following Lindeberg's time-causal, time-recursive temporal scale-space theory, a cascade of first-order recursive filters with geometrically spaced time constants is the canonical way to represent a signal simultaneously at many temporal scales when (i) future values may not be accessed and (ii) no memory buffer beyond the representation itself is permitted. LTSL is useful for complex data where timescale of temporal dependencies in many such applications is unknown. Geometric (logarithmic) spacing is what makes the family scale-covariant: a temporal rescaling of the input shifts the representation along the scale axis instead of destroying it. For financial markets, whose characteristic time-scales are neither fixed nor known, that is an attractive prior.

The contributions documented in this report are:

- A concrete coupling of the LTSL to a BCPNN (see Section 3): a TDPop population whose K hypercolumns carry successively slower cascade stages of the encoded input, projecting through a single Bayesian–Hebbian projection onto a softmax output population that represents the predictive pmf.
- A continual, walk-forward **multislice** training schedule (Section 4) in which one simulator process sweeps a shared calendar in blocks, concatenating every instrument, and never resets weights or filter state, resulting in exceptional speed.
- The empirical route from Mackey–Glass to equities to FX (Section 5), including the volatility-clustering analysis that motivated replacing the (mean, first derivative) feature set with (log return, trailing variance).
- Important lessons from the code development that include the diagnosis and repair of three defects that biased the predictive tail, verified on a synthetic Gaussian probe with a known 1 % quantile (Section 6).
- An evolutionary hyperparameter search directly on the tick loss (Section 7), and the finding that near-optimal genotypes unanimously retain logarithmic ladder spacing and linear-spaced RBF centres.
- A head-to-head benchmark against DeepVaR on identical data and an identical backtest window, with coverage and independence tests (Section 8–9), including a critique of two of the four published loss definitions.

2 Background

2.1 The BCPNN learning rule

BCPNN starts from naïve Bayes and moves the inference into log space so that evidence combines additively and can be propagated by an ordinary weighted sum, resulting in a data architecture that is very native to neural networks. For a presynaptic unit i and a postsynaptic unit j with estimated marginal activation probabilities P_i , P_j and joint probability P_{ji} , the weight and the intrinsic bias are

$$W_{ji} = \log \left(\frac{P_{ji}}{P_i \cdot P_j} \right), \quad B_j = \log P_j$$

so that the weight is the pointwise mutual information between the two units and the bias is a learned prior of the postsynaptic unit. Positively correlated units acquire excitatory weights, anticorrelated units inhibitory ones, and independent units exactly zero — the rule is Hebbian, local, and derived rather than postulated.

The probabilities are estimated online by exponentially weighted traces. In the implementation used here each activation first enters a fast Z-trace with an ε -floor, and the Z-traces then drive slow P-traces with time constant τ_p :

$$Z \leftarrow Z + (f \cdot a \cdot (1 - \varepsilon) + \varepsilon - Z) \frac{\Delta t}{\tau_z}$$

$$P \leftarrow P + (Z - P) \cdot \kappa \cdot \frac{\Delta t}{\tau_p}$$

The learning gate κ is 1 during training and 0 during evaluation, which is how the same process can alternate between learning and frozen recall without copying weights. The floor ε is a Laplace-style regulariser that avoids log 0 on bins that have never been sampled. Units are grouped into hypercolumns of M minicolumns and normalised by a softmax within each hypercolumn, so hypercolumn activity is a probability distribution and the whole network is a soft winner-take-all mosaic, matching some conceptions of encoding in cortex, primary sensory cortex in particular.

2.2 Time-causal temporal scale-space and the logarithmic ladder

Lindeberg's temporal scale-space theory asks which convolution kernels may be used to smooth a signal over time such that structure is never created as scale increases, while respecting two hard constraints: the kernel must vanish for future arguments (time-causality) and the representation at the next instant must be computable from the representation itself (time-recursivity). The classification result is sharp: over a continuous domain the admissible time-causal scale-space kernels are **truncated exponentials coupled in cascade**, and their discrete analogue is a cascade of first-order recursive filters. The Gaussian, which is the canonical smoothing kernel over space has to be excluded because it reaches into the future.

Adding a requirement of temporal scale covariance — that rescaling the input in time should merely shift the representation along the scale axis — singles out a self-similar, geometric distribution of the time constants and, in the limit of infinitely many stages, the *time-causal limit kernel*. This is the theoretical justification for the logarithmic ladder: with a fixed budget of K kernels, geometric spacing gives constant *relative* resolution across the whole range of scales, whereas linear spacing squanders most of the budget on nearly redundant slow kernels. Because the cascade is recursive, the entire multi-scale memory of the past is carried in $K \cdot M$ floating-point state variables, with no ring buffer and no windowing.

2.3 Why the two fit together

The pairing is natural in both directions. The LTSL needs a downstream reader that can exploit a high-dimensional, redundant, multi-scale code; BCPNN's naïve-Bayes weight rule handles exactly that, because each ladder rung (temporal stage) contributes an independent log-likelihood term and the softmax then renormalises the accumulated evidence. Conversely, BCPNN is an associative memory with no intrinsic notion of time; the LTSL supplies the temporal receptive fields it lacks. The result is an ultra-fast model with no backpropagation, no gradient steps and no epochs: one forward pass per observation updates the traces, and by simply closing the learning gate, the same pass yields a prediction that naturally handles uncertainty through its pmf representation.

3 Model and implementation

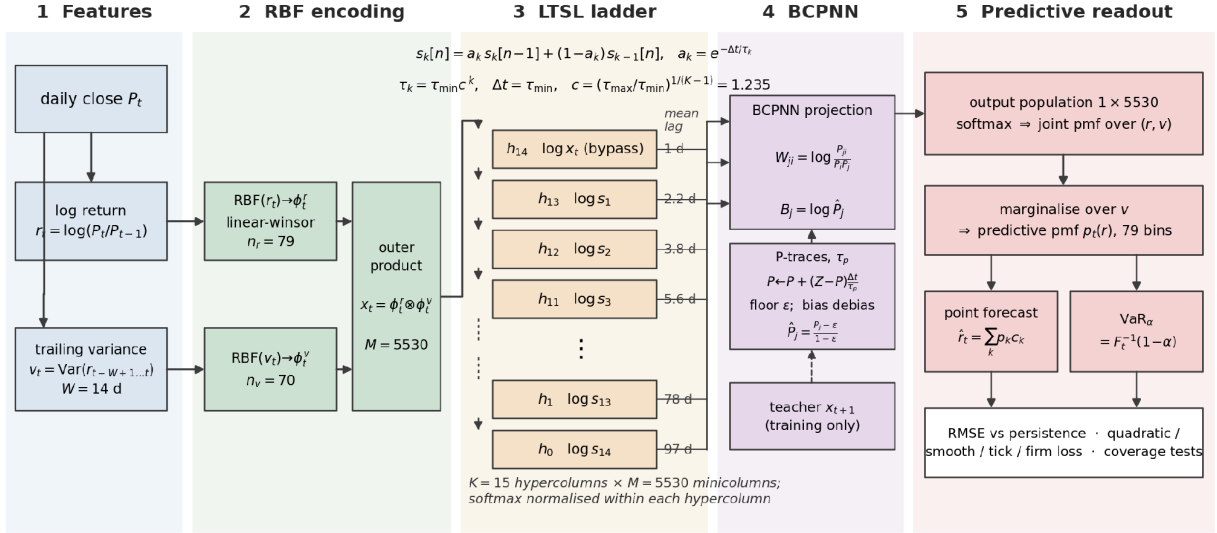


Figure 1. End-to-end architecture. Daily closes are reduced to a log return and a trailing return variance, each encoded by an independent one-dimensional RBF grid; the outer product forms a single $M = 5530$ -dimensional input pattern. The LTSL cascade exposes $K = 15$ hypercolumns — one instantaneous bypass channel plus fourteen progressively slower cascade stages — which project through one Bayesian–Hebbian projection onto a softmax output population. Marginalising the output over the variance axis yields the predictive pmf over next-day log return, from which both the point forecast and the VaR quantile are read.

3.1 The LTSL population

The ladder is implemented as a “TDPop”, a population of $H = K$ hypercolumns each holding M minicolumns. For every minicolumn m the population runs a cascade of H first-order lowpass filters,

$$s_0[n] = a_0 s_0[n-1] + (1 - a_0) x_m[n]$$

$$s_k[n] = a_k s_k[n-1] + (1 - a_k) s_{k-1}[n], \quad \text{where } a_k = e^{-\Delta t / \tau_k}$$

In logarithmic mode the time constants (see Figure 2) are

$$\tau_k = \tau_{\min} \cdot c^k, \quad \text{where } c = \left(\frac{\tau_{\max}}{\tau_{\min}} \right)^{\frac{1}{K-1}}$$

In linear mode time constants are equally spaced. Two implementation details matter for interpretation. First, the driver sets the sampling rate to $f_s = 1/\tau_{\min}$, so $\Delta t = \tau_{\min}$: the ladder is scale-free in $\tau_{\min}$. So the ratio $c = \tau_{\max}/\tau_{\min}$ and the temporal stage count K together tune the LTSL. Expressed in pattern steps (days for the financial data we apply this to) the per-stage constant is simply c^k . Second, because the best representation of the fastest data component is the raw unfiltered input, stage 0 is computed but not exposed. The last hypercolumn instead carries the log of the raw input, so the population holds one instantaneous bypass channel and $K-1$ filtered channels:

$$\text{lg}[h] = \log s_{K-1-h} \quad \text{for } h = 0 \dots K-2, \quad \text{lg}[K-1] = \log x$$

This also means that the last hypercolumn holds special significance for the readout of an interpretable pmf. Because stage k is the cascade of stages $0 \dots k$, its composite kernel has mean lag $\sum_{j \leq k} c^j$ steps and standard deviation $(\sum_{j \leq k} c^{2j})^{1/2}$. For the tuned FOREX configuration ($K = 15$, $\tau_{\min} = 39.125$, $\tau_{\max} = 751.86$, $c = 1.2351$) the exposed channels therefore span mean lags from 1 to about 97 trading days, i.e. covering data from the last day to 4.5 months ago, with progressively broader kernels (Table 1 and Figure 2). Each hypercolumn is softmax-normalised independently.

Table 1. Effective temporal receptive fields of the tuned FOREX ladder. The internal τ values are only meaningful relative to $\tau_{\min}$ because the simulator sets $\Delta t = \tau_{\min}$

hypercolumn h	cascade stage	r_k (internal)	c^k (steps)	mean lag (days)	kernel s.d. (days)
14 (bypass)	—	—	—	1.0	0
13	s_1	48.3	1.24	2.2	1.6
12	s_2	59.7	1.53	3.8	2.2
11	s_3	73.7	1.88	5.6	2.9
...	...	...	...	...	...
1	s_{13}	608.8	15.56	77.5	26.5
0	s_{14}	751.9	19.22	96.7	32.7

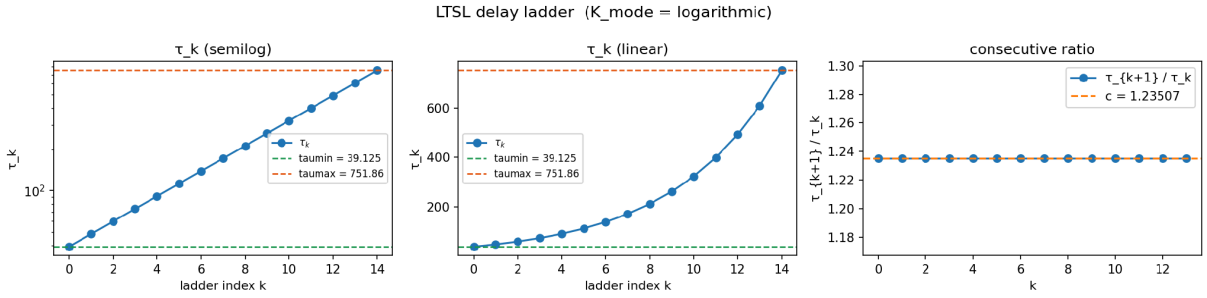


Figure 2. The tuned ladder: fifteen time constants, geometric on a semilog axis (left), strongly convex on a linear axis (centre), with a constant consecutive ratio $c = 1.235$ (right). Note that if linear spacing were used instead, the ladder would dedicate almost its entire computational budget to redundant, very slow filters while severely under-sampling the rapid, short-term dynamics relevant to the next-day prediction. Geometric spacing (constant consecutive ratio) is what preserves scale covariance across the network's multi-scale memory.

3.2 Learning and recall in the projection

A single projection connects the whole ladder to the output population, allowing the model to efficiently discover and encode temporal correlation at any of the provided scales. During training, the output population is clamped to the pattern observed offs steps later ($\text{igain} = 1$, projection gain 0) while the traces integrate; during recall the clamp is released and the projection gain is set to 1, so the output is generated purely from the ladder state. The P -trace time constant defaults to the length of the training stream ($\tau_p = \text{trnpat} \cdot \Delta t$, i.e. 13 024 pattern steps for the FOREX run) and the floor defaults to $\varepsilon = 1/(1+\text{trnpat}) = 7.7 \times 10^{-5}$. In other words the network accumulates its Bayesian statistics over an exponential window as long as the training set itself, which is what makes the walk-forward schedule (see Section 4) a genuine continual learner rather than a sequence of independent fits.

One correction to the textbook rule is applied. The ε -floor in the Z-update injects a uniform mass of order $M \cdot \varepsilon$ into every learned marginal. That is harmless for the weights, where it acts as the intended regulariser, but it inflates the learned prior B_j and thereby flattens the recalled pmf — precisely the wrong direction for a long tail quantile we are concerned with here. Enabling the new parameter `bdebias` inverts the floor analytically **in the bias term only**:

$$\hat{P}_j = \max \left(\frac{P_j - \varepsilon}{1 - \varepsilon}, \varepsilon^2 \right), \quad B_j = \log \hat{P}_j$$

Weights keep the floored estimates, so rare coincidences remain regularised while the prior admixture is removed. Section 6 shows what this is worth on a distribution whose true quantiles are known.

3.3 Features

Each instrument is max-normalised and reduced to four columns, of which two are used (later code releases will remove unnecessary features). In the current feature set (`return_variance`) the **primary** channel — the prediction target — is the daily log return $r_t = \log(P_t/P_{t-1})$, and the **secondary** channel is a left-sided, direction-agnostic trailing variance of the log returns over a window W (14 days after tuning). The secondary channel conditions the prediction on the current volatility regime without leaking any information about direction, and it is strictly causal. The earlier feature set (`mean_derivative`) instead predicted a seven-day trailing mean of the normalised price with a first derivative (Fornberg endpoint finite-difference weights) as the side channel; Section 5 explains why it was abandoned.

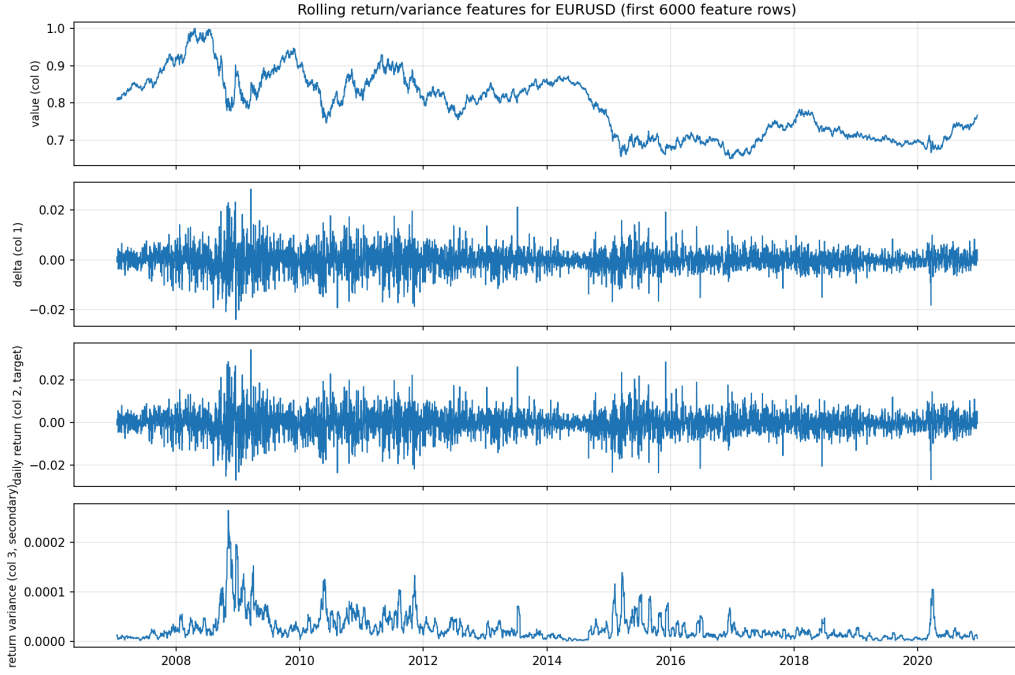


Figure 3. EUR/USD feature channels over the full 2007–2020 calendar. Top to bottom: max-normalised close, simple price delta, the daily log return that the network predicts, and the 14-day trailing return variance that conditions it. The variance channel is visibly regime-like — the 2008 crisis, the 2015 EUR devaluation and the March 2020 COVID shock each produce a sustained excursion — whereas the return channel looks like noise with a time-varying amplitude.

3.4 Encoding and decoding

Both channels are encoded by independent one-dimensional Gaussian radial-basis functions and normalised to a distribution over bins,

$$\phi_k(x) = \frac{\exp\left(-\frac{1}{2}\left(\frac{x-c_k}{\sigma_k}\right)^2\right)}{\sum_j \exp\left(-\frac{1}{2}\left(\frac{x-c_j}{\sigma_j}\right)^2\right)}$$

Two centre layouts are available in the current code base. `binmode = rbf` places n centres linearly across the 0.25 %/99.75 % winsorised range with σ equal to one centre spacing; `binmode = rbf-quantile` places them at the $(i+\frac{1}{2})/n$ empirical quantiles with a locally adaptive σ . Our analysis revealed that the distinction is decisive for tail work and is analysed in Section 6.3. The choice to windsor/clip away only 0.25% in rbf mode was made to preserve the integrity of the outer 1% quantile we need for VaR estimation at the 99% level.

The two encodings are combined by an outer product,

$$x_t = \phi_t^r \otimes \phi_t^v \in \mathbb{R}^{n_r \times n_v}$$

giving a single $M = 79 \times 70 = 5\,530$ -dimensional pattern per day, written to `features.dat` as float32. The outer product is what lets a purely associative network represent a *conditional* distribution: a joint code over (return, variance) means the learned weights encode $P(\text{return} \mid \text{volatility regime})$ implicitly.

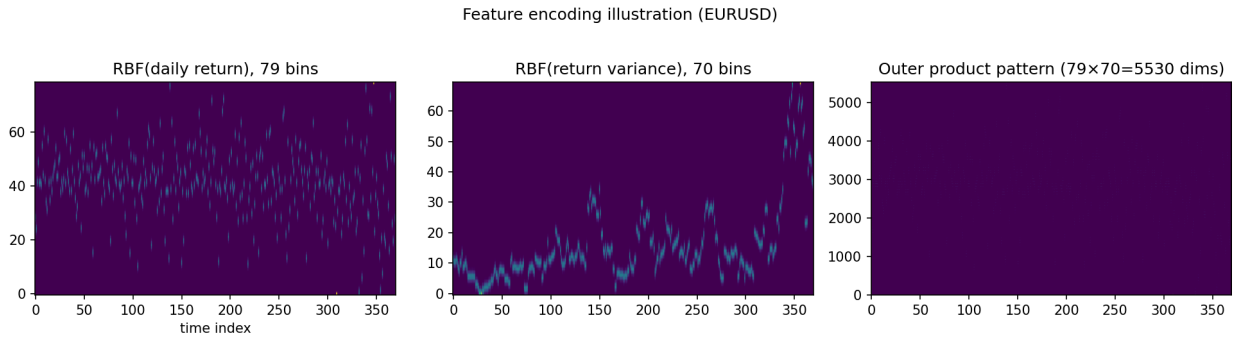


Figure 4. Encoding of a 370-day EUR/USD slice. Left: the 79-bin return code, which wanders around the centre of the grid. Centre: the 70-bin variance code, which drifts slowly and coherently — the volatility regime is the part of the signal with structure. Right: the resulting 5 530-dimensional outer-product pattern stream.

Decoding inverts the encoding. The simulator marginalises the joint output activity over the variance axis and writes a pmf over the 79 return bins for every step. From that pmf the point forecast is the centroid $\sum_k p_k c_k$, and the VaR at level α is the discrete inverse CDF evaluated at $1-\alpha$, obtained by linear interpolation on the cumulative distribution over bin centres. Ensuring that the floor sits below the quantile of interest is a design constraint on the encoder. Because `np.interp` clamps below the first centre, the leftmost bin centre acts as a hard **floor** on the reportable VaR; for the pooled four-pair FOREX grid that floor is -0.0264 , comfortably deeper than any VaR the model actually reports (mean -0.0119).

3.5 Software pipeline

The system is split between a Python front end and a more efficient CUDA/C++ simulator (we'll be releasing a python interface to the LTSL version of that simulator shortly). Python (`FeatureGeneration.py`, `RBFBinners.py`, `MultislicePlan.py`) loads the wide CSV, computes features, fits the encoders on the pooled stream, writes `features.dat`, the centre sidecar `features.centers.npy` and the block schedule `schedule.dat`. The compiled simulator `ltslmain` (built on the BCPNN library in `libsrc_cu/`) reads those three files, runs the whole schedule inside a single process, and streams the output population activity to `netw1_outpop_*_act.bin`, marginalising it into `*_dist.bin`. Python then reloads the pmf streams, un-stitches them per instrument, and computes RMSE, persistence baselines, VaR and the DeepVaR loss suite (`MultisliceAnalyze.py`, `DeepVaRLosses.py`). The full four-pair FOREX run is 30452 simulator steps and takes 744 s on one GPU (NVIDIA RTX A4000); re-running it reproduces every reported figure bit-for-bit, as the model is deterministic given the parameter file.

Table 2. The notebooks that drive the pipeline, in the order the project developed.

notebook	role	data	feature set
general/notebook-MG_offs1.ipynb	proof of concept, horizon and ladder sweeps	Mackey–Glass	mean + 1st deriv.
nb-multislice_mean_1stderiv.ipynb	first multi-instrument walk-forward study	S&P 100, 98 tickers	mean + 1st deriv.
nb-multislice_return_variance.ipynb	feature-set switch, 72 h EA search	S&P 100, 98 tickers	return + variance
nb-1ticker.ipynb	single-instrument pipeline check	S&P 100, NFLX	return + variance
nb-SYNTH.ipynb	calibration probe with known 1 % quantile	synthetic Gaussian	return + variance
nb-FOREX.ipynb	main study, EA post-mortem, DeepVaR benchmark	4 FX pairs	return + variance

4 The multislice continual-learning schedule

Holding out entire instruments/tickers for testing leaks calendar structure: a held-out ticker spans the same crises as the training tickers, so the model can have memorised the regime even if it has never seen that symbol. Our **multislice** protocol avoids this by cutting the *shared calendar* instead. Block b covers feature rows $[W-1+b\Delta, W-1+(b+1)\Delta]$ where $\Delta = \text{trnpat} + \text{vanpat} + \text{tenpat}$, and within a block the patterns of every active instrument are concatenated phase-major and ticker-minor: all training slices first, then all validation slices, then all test slices. One row of `schedule.dat` per block gives the simulator the per-phase totals. As some stocks might have later IPOs (some missing data in the first blocks), each block only includes stocks that have complete data in that blocks time range.



Figure 5. The multislice schedule. (A) The shared calendar is cut into consecutive blocks; every instrument contributes a train, validation and test slice to each block it has sufficient history for, so late listings simply sit out early blocks. (B) The resulting pattern stream is phase-major and

ticker-minor and is consumed by a single simulator process — weights and LTSL filter states are never reset, so block index doubles as a measure of how much history the network has already learned from.

The consequences are worth stating explicitly. Because everything runs in one process, the P -traces and the K - M filter states carry across both instrument boundaries and block boundaries. The network is therefore a single continual learner accumulating a general market model, not an ensemble of per-symbol fits; a later block is evaluated by a network that has already been trained on all earlier blocks and all earlier instruments. Autoregressive generation that was so useful in evaluating the stability of Mackey-Glass prediction during early development (`general/notebook-MG_offs1.ipynb`) is disabled in this mode (`ngenstep = 0`), because free-running across a multi-instrument boundary is ill-defined.

Splicing two unrelated instruments back to back does inject a transient into the ladder. The optional `warmup_pat` prefix handles this: at every ticker-to-ticker cut after the first, the leading patterns of the segment are replayed forward-only, with weights frozen and logging suppressed, so scoring starts only once the filters have settled. The S&P 100 runs use 50–100 warm-up steps; the FOREX run uses none, because with only four pairs and 3 256 training rows each the transient is a negligible fraction of the stream.

The S&P 100 studies use six blocks and 98 instruments, with late listings joining progressively as their history becomes sufficient (81 active tickers in block 0, 98 in block 5). For the FOREX study all four pairs share one full-history block of 4 358 feature rows, split 3 256 / 20 / 1 082 into training (2007-01-16 – 2017-06-11), validation, and test (2017-07-06 – 2020-12-18), giving per-block totals of 13 024 / 80 / 4 328 patterns. 5 From chaotic dynamics to markets

5.1 Mackey–Glass: does the ladder carry enough information?

The first question was whether a BCPNN reading an LTSL can predict a time series at all. The Mackey–Glass delay-differential system ($\beta = 0.2$, $\gamma = 0.1$, $n = 10$, $\tau = 17$, $\Delta t = 1$, integrated by explicit Euler with a 500-step transient discarded and max-normalised) is the standard chaotic benchmark for this. With $K = 12$ linearly spaced kernels, a 16×18 encoding and 10 840 training patterns, one-step prediction of the seven-day trailing-mean channel reaches RMSE 0.0212 on training data, 0.0207 on validation and 0.0221 on unseen data, against a persistence baseline of 0.0233. At a ten-step horizon the errors rise to about 0.0636 and the model no longer separates cleanly from persistence (0.2031 versus 0.2017 in free-running generation). A sweep over the temporal stage count found the error still improving at $K = 18$ (0.0204), and — notably for what follows — linear spacing beat logarithmic spacing on this signal, which has a single characteristic time-scale set by its delay parameter. Free-running autoregressive generation was poor (0.246 at a one-step horizon), which is expected for a chaotic attractor and was not pursued further.

The conclusion drawn was modest but sufficient: the LTSL code carries enough temporal information for a Bayesian associative reader to beat persistence on a deterministic chaotic signal, and the machinery runs at scale.

5.2 S&P 100 with mean and first-derivative features

For a more challenging dataset, the pipeline was then applied to a stock market index (S&P 100) with 98 constituents (2000-01-03 to 2025-02-24, 6 324 calendar rows) in six walk-forward blocks of 750 / 52 / 200 patterns per instrument, with 50 warm-up steps at each ticker cut — 543 084 patterns and 947 416 simulator steps in a single 800 s run. Aggregate test RMSE on the trailing-mean channel was 0.0145 against a persistence baseline of 0.0018. Cross-Section ally the ordering was exactly what a market practitioner would predict — defensive staples such as PG (0.0043) and KO (0.0046) are far ahead of high-beta names such as AAPL (0.078) and AIG (0.073) — and the worst single blocks clustered on crisis periods.

But the model never beat persistence, for any of the 98 tickers. That is not a failure of learning; it is a statement about the target. A seven-day trailing mean of a max-normalised price is almost perfectly autocorrelated at a one-day horizon, so $y_{t+1} \approx y_t$ is an extremely strong baseline that no distributional model over 100 coarse bins can match. The experiment was measuring quantisation error, not forecasting skill.

A high-resolution lag-1 transition analysis over 582 957 pooled transitions then made the diagnosis precise. The first-derivative side channel is strongly volatility-persistent — the autocorrelation of $|\Delta|$ averages **0.814** and is positive for every single ticker — but strongly **sign-reverting**: the probability that a large move is followed by a move of the same sign is only **0.066**. The side channel therefore carried a large, exploitable magnitude signal that the model was not being asked to use, while the target channel it *was* being asked to predict was nearly redundant with its own last value. As we have not worked on financial data before this was a very important learning for us that demanded consequences.

5.3 Switching to return and variance

The feature set was therefore restructured around that asymmetry: predict the signed daily **return**, and condition on a direction-agnostic trailing **variance**. Repeating the same 98-ticker, six-block protocol with the new features ($K = 14$, 4×20 bins, $W = 19$, 500 / 250 / 250 per block) gives aggregate test RMSE 0.0165 against a persistence baseline of

0.0236 — and now **all 98 tickers beat persistence**, in **every** block, clearly demonstrating how important it is to pick the right features. The run is also four times cheaper (213 s), because the joint pattern is 80 dimensions instead of 500.

Repeating the transition analysis on the new target confirms the reading: the daily return channel has $|\Delta|$ autocorrelation **0.265** (positive for essentially all tickers) but sign persistence **0.482**, i.e. indistinguishable from a coin flip, yet slightly reverting on very large market moves, consistent with trading strategies aimed at profit-taking at highs and buying-cheap at lows. Magnitude is predictable; direction is not (except for rare extreme moves), see Figure 6.

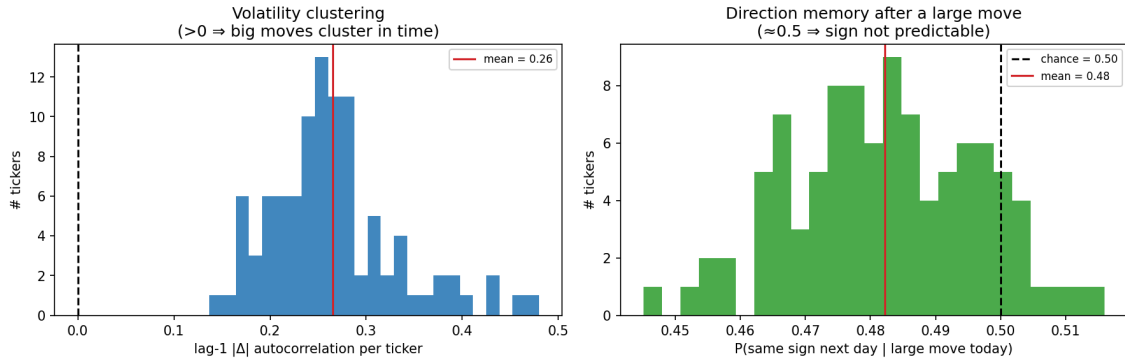


Figure 6. Predictability of the daily-return channel, pooled over 98 S&P 100 constituents. Volatility autocorrelation is positive for every ticker (mean 0.265) while the probability that a large move is followed by a same-signed move sits at 0.482, i.e. chance. This is the empirical basis for treating the problem as a distributional one.

A 72-hour evolutionary search over seven parameters (1 341 evaluations, 635 unique genotypes, best validation RMSE 0.01770) then delivered a result that is unintuitive until read alongside the transition analysis: near-optimal genotypes cluster at **coarse** return grids (2–5 bins) and **fine** variance grids (15–20 bins), the opposite of the naïve allocation. Test RMSE is essentially flat in return_nbin from 2 to 20. Since scoring is done on the decoded continuous return and not on bin indices, this is not a metric artefact: the network simply does not need resolution on a quantity it cannot forecast, and does need it on the volatility regime it can. Every one of the 522 unique near-optimal genotypes used quantile centres — a preference that reverses once the objective changes from RMSE to a tail loss (Section 6.3, Section 7).

5.4 The FOREX dataset

The final study uses the dataset of the DeepVaR paper: daily closes for AUDUSD, GBPUSD, JPYUSD and EURUSD from 2007-01-01 to 2020-12-18, 4 371 calendar rows. With a 14-day variance window each pair yields 4 358 feature rows, exactly one multislice block, split 3 256 training / 20 validation / 1 082 test rows per pair (13 024 / 80 / 4 328 patterns in total). Training ends 2017-06-11 and the test phase covers 2017-07-06 to 2020-12-18, 1 081 scored steps per pair.

6 Calibrating the predictive tail

Estimating a 99 % VaR is a much sharper test of a probabilistic model than estimating a mean: it depends on a region of the output distribution that is, by construction, visited on one day in a hundred. Three defects that were invisible under an RMSE objective turned out to dominate the tail early on and had to be addressed to achieve a well calibrated model.

6.1 Three defects

The RBF width was not scale-invariant. `from_linear_winsor` defaulted to $\sigma = 1/n$ in absolute data units. For a price channel normalised to $[0,1]$ that is one bin spacing, as intended; for a log-return channel whose winsorised range is about 0.021 wide it is roughly the entire range, so every kernel overlapped every other and the encoding collapsed to near-uniform (normalised entropy above 0.997). The network dutifully learned a flat pmf. The default is now $\sigma = (hi-lo)/n$.

The regularisation floor never reached the projection. The parameter file's `eps` was parsed, printed and then discarded: the projection silently kept the library default of 10^{-7} . Rarely visited joint bins therefore accumulated multi-nat sampling noise in their log-odds, and the softmax read-out sprouted spurious peaks in the outermost bins — 54 % of synthetic test steps peaked in the outer 10 of 50 bins. Wiring seteps through drops that to about 9 %.

The floor then biased the prior. With a real ϵ in place, the Z-trace floor adds roughly $M \cdot \epsilon$ of uniform mass to the learned marginal — about a 40 % uniform admixture at the synthetic run's default — which compressed the recalled bell and made edge bins four to five times too heavy. `bdebias` (Section 3.2) removes exactly this term from the bias

while leaving the weights regularised. The three fixes are complementary: before ϵ was wired, debiasing a floor of 10^{-7} would have been meaningless.

6.2 A probe with a known answer

To separate estimator bias from market non-stationarity, nb-SYNTH.ipynb builds a price series whose return law is known exactly: 5 000 i.i.d. draws from $N(0, \sigma)$ with $\sigma = 0.01/z_{0.99} = 0.0042986$, chosen so that exactly 98 % of the mass lies in $[-0.01, +0.01]$ and the true 1 % quantile is therefore exactly -0.01 . The series is written to a CSV with the same structure as the FX data and run through the identical pipeline (3 900 / 100 / 990 split, 989 scored test steps, so 9.89 violations are expected at the nominal rate).

After the three fixes the model reproduces the target law closely: the mean of the predicted pmf is -0.000117 against a target of 0, the mass inside the band is 0.9811 against 0.98, and the backtest records **10 violations in 989 steps (1.011 %, *z* = +0.04)** — statistically indistinguishable from **perfect calibration**. Mean predicted VaR is -0.00967 against a true -0.01000 , a discrepancy explained entirely by grid geometry rather than by bias (Section 6.3).

Comparing the average predicted test pmf against the true Gaussian discretised on the same quantile grid, both sit at the $1/50 = 0.02$ level across every bin — quantile-spaced bins carry equal mass by construction — with only finite-sample jitter between them. Before the fixes the same comparison showed edge bins four to five times too heavy from legacy implementation of BCPNN that were simply not intended to care about deep calibration of long probability tails.

6.3 Bin geometry decides what the tail can express

With quantile centres and n bins the leftmost centre sits at the $(0.5/n)$ quantile — for $n = 50$ that is the 1 % quantile itself, so VaR_{99} lands exactly on the edge of the representable grid and cannot be reported any deeper. Linear winsorised centres instead spread evenly across the 0.5 %/99.5 % range, which for this Gaussian puts the leftmost centre at -0.0106 , *beyond* the true quantile.

On the synthetic Gaussian this favours quantile centres: they hit 1.011 % coverage ($z = +0.04$) while linear centres are over-conservative at 0.506 % ($z = -1.56$), placing 3.96 % of mass below -0.01 against a target of 1 %. On real FX the preference reverses, and the evolutionary search of Section 7 selects linear centres unanimously. The two results are consistent rather than contradictory: quantile centres are efficient when the law is light-tailed and stationary, because they spend resolution where the mass is; FX returns are heavier-tailed and non-stationary, so what matters is having grid headroom *beyond* the quantile being scored, and the pooled quantile grid is fitted to an unconditional sample that under-represents crisis days. We report both results, because the synthetic probe is the cleaner experiment and the FX preference is an empirical, objective-dependent finding rather than a general law.

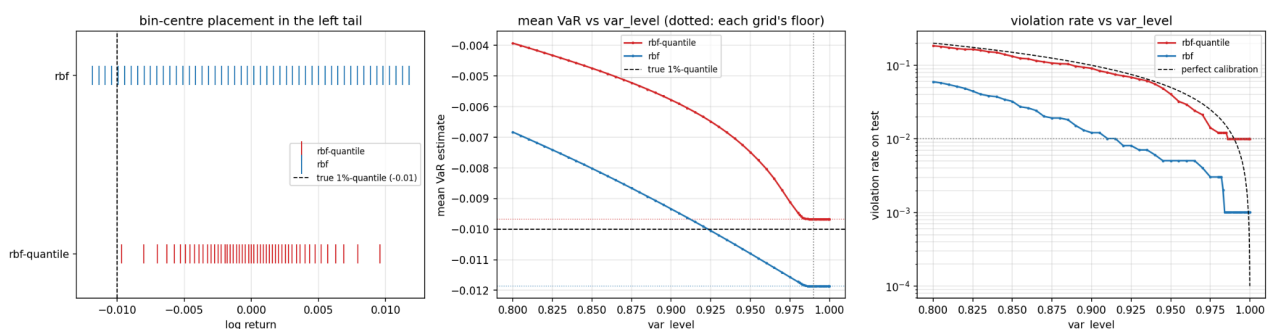


Figure 7. Bin geometry on the synthetic Gaussian. Left: the leftmost quantile centre sits almost exactly at the true 1 % quantile, whereas the linear grid reaches past it. Middle+Right (log plot): mean reported VaR and empirical violation rate as a function of the requested level; both curves saturate at their grid's floor, which is why the leftmost centre must be placed deeper than the quantile of interest.

7 Nine-dimensional Hyperparameter search and 1D Sensitivity Analysis

Nine parameters were searched by a generational evolutionary algorithm (evolve_hyperparams_v4.py; population 24, tournament selection with $k = 3$, uniform crossover, per-gene mutation probability 0.25, two elites, wall-clock termination, memoised on the genotype). Each evaluation is a full multislice run of the four pairs, so the search cost is dominated by the simulator: **318 evaluations, 299 unique genotypes, 15 generations, 47.6 hours**. The objective was the DeepVaR tick loss at $\alpha = 0.99$, averaged over pairs.

Table 3. Search space and outcome. The near-optimal band is the 35 unique genotypes within 0.5 % of the best tick loss (1.5877×10^{-4}).

parameter	search range	EA optimum	near-optimal band
K	2 – 20	15	median 15
K_mode	{linear, logarithmic}	logarithmic	logarithmic in 35/35

return_nbin	2 – 150	79	median 70
variance_nbin	2 – 100	70	median 70
variance_window	3 – 60	14	median 14
binmode	{rbf, rbf-quantile}	rbf	rbf in 35/35
taumin	1 – 200	39.13	—
taumax	100 – 1000	751.86	median 843
bdebias	{0, 1}	1	1 throughout

The search recorded fourteen new-best events and was still improving slowly at the budget limit, though the last three improvements were worth less than 1 % each. Two structural results survive the noise. First, **every** genotype in the near-optimal band uses logarithmic ladder spacing, and **every** one uses linear RBF centres. Given that the search was free to choose linear spacing and quantile centres — and that the S&P 100 RMSE search of Section 5.3 chose quantile centres unanimously — this is a strong, objective-specific signal: tail scoring wants grid headroom in the left tail and constant relative temporal resolution. Second, the best genotype's violation rate is 0.996 % against a 1 % target, i.e. the tick loss really is behaving as a calibration objective rather than as a proxy for something else.

Point-forecast accuracy and tail calibration are only weakly coupled: across all evaluated genotypes the Pearson correlation between tick loss and validation RMSE is $r = -0.29$, and a twelve-point Pareto front separates the two objectives. The RMSE-optimal genotype achieves a violation rate of 0.269 %, badly over-conservative. RMSE is therefore a useful sanity check but a poor surrogate objective for risk.

One-dimensional sweeps around the optimum confirm that it is locally optimal in every coordinate. The most informative single result is the encoder comparison: linear RBF centres give tick loss 1.583×10^{-4} against 2.142×10^{-4} for quantile centres, a 26 % gap.

Table 4. One-dimensional sensitivity around the EA optimum, scored by tick loss at $\alpha = 0.99$ on the FOREX test phase.

swept parameter	values	best	tick loss at best	worst in sweep
binmode	rbf, rbf-quantile	rbf	1.583×10^{-4}	2.142×10^{-4}
taumin	5, 10, 20, 40, 60, 80	20	1.578×10^{-4}	1.619×10^{-4}
taumax	100, 500, 750, 1000, 1500	1500	1.578×10^{-4}	1.715×10^{-4}
return_nbin	20, 40, 79, 100, 150	79	1.583×10^{-4}	1.818×10^{-4}
variance_nbin	20, 40, 70, 90, 110	70	1.583×10^{-4}	1.730×10^{-4}
variance_window	7, 14, 21, 28	14	1.587×10^{-4}	1.639×10^{-4}

For completeness: an earlier search run under the **incorrect** tick-loss polarity (Section 9.2) and before the three fixes of Section 6 converged on a spurious optimum with a 21.8 % violation rate and a nominal tick loss of 3.5×10^{-3} . None of its conclusions — which favoured quantile centres, linear spacing and a short $T_{\max}$ — survived, showing that the computational advantage of BCPNN-LTSL reveals itself only with significant implementational attention to encoding and proper calibration.

8 Results on FOREX

8.1 Point forecasts, and an honest baseline

Aggregated over the four pairs, test RMSE on the daily return is 0.004685 against a persistence baseline of 0.006522, a 28 % improvement, and every pair beats persistence in every phase. That number should not be over-read. Persistence — predicting that tomorrow's return equals today's — is a weak baseline for a nearly serially uncorrelated series, since it inflates the error by roughly $\sqrt{2}$. The relevant benchmark is the unconditional mean, and against that the model has **no measurable edge at all**, highlighting how incredibly hard point forecasting financial time series is.

Table 5. Point-forecast quality on the 1 081-step test phase. The model matches, but does not beat, a constant forecast at the unconditional mean, and the directional hit ratio is indistinguishable from 50 %.

pair	test RMSE	predict-mean RMSE	persistence RMSE	model / predict-mean	directional hit
AUDUSD	0.005439	0.005454	0.007520	0.997	51.3 %
GBPUSD	0.005272	0.005247	0.007234	1.005	47.6 %
JPYUSD	0.004154	0.004113	0.005929	1.010	51.5 %
EURUSD	0.003873	0.003845	0.005410	1.007	48.0 %
mean	0.004685	0.004665	0.006523	1.005	49.6 %

This is the expected outcome given Section 5.2–5.3, and it is worth stating plainly rather than hiding behind the persistence comparison: **the value of this model is not in its conditional mean**. What it provides is a conditional

distribution whose scale and shape track the volatility regime, and that is what the remainder of this Section evaluates.

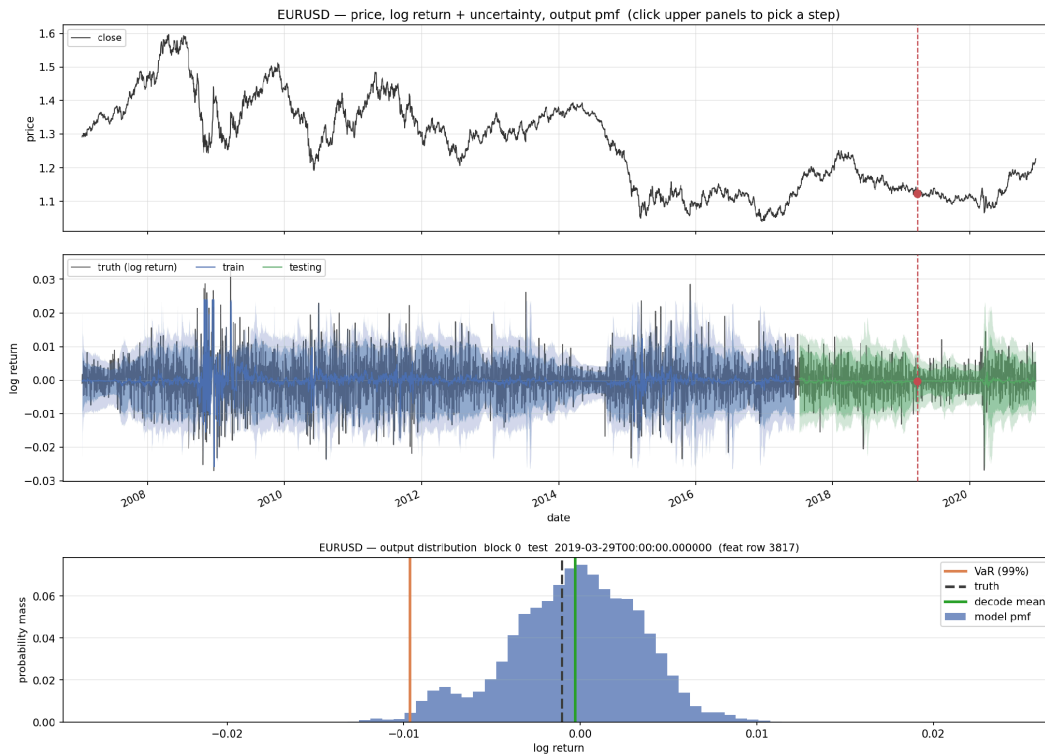


Figure 9. EUR/USD walk-forward output. Top: the price path with the inspected day marked. Middle: realised log return (black) with the model's 90 % and 99 % credible bands, blue during training and green during the test phase; the bands widen and contract with the volatility regime, including the visible March 2020 expansion. Bottom: the full predictive pmf for one day, with the decoded mean, the realised return and the 99 % VaR quantile.

8.2 Value-at-Risk backtest

VaR_{99} is read from the same pmf as the left 1 % quantile. Over the 1 081-step test phase the four pairs record 11, 14, 12 and 8 violations, i.e. rates between 0.74 % and 1.30 % against a nominal 1 %, and 45 of 4 324 pooled steps (1.04%, **statistically indistinguishable from perfect calibration**). All four pairs pass Kupiec's unconditional-coverage test, Christoffersen's independence test and the joint conditional-coverage test at any conventional level.

Table 6. Full test-phase backtest at $\alpha = 0.99$, 2017-07-06 to 2020-12-18, 1 081 steps per pair. No test statistic approaches its 5 % critical value (3.84 for one degree of freedom, 5.99 for two).

pair	viol.	rate	mean VaR	quad.	smooth	tick $\times 10^{-4}$	firm	LR_{uc}	LR_{ind}	LR_{cc}
AUDUSD	11	1.02 %	-0.01311	0.01018	0.00773	1.83	0.01298	0.00	0.23	0.23
GBPUSD	14	1.30 %	-0.01359	0.01295	0.00805	1.79	0.01341	0.87	1.88	2.75
JPYUSD	12	1.11 %	-0.01086	0.01110	0.00629	1.45	0.01074	0.13	2.43	2.55
EURUSD	8	0.74 %	-0.01033	0.00740	0.00591	1.26	0.01025	0.81	0.12	0.93
pooled	45	1.04 %	—	—	—	1.62	—	0.07	—	—

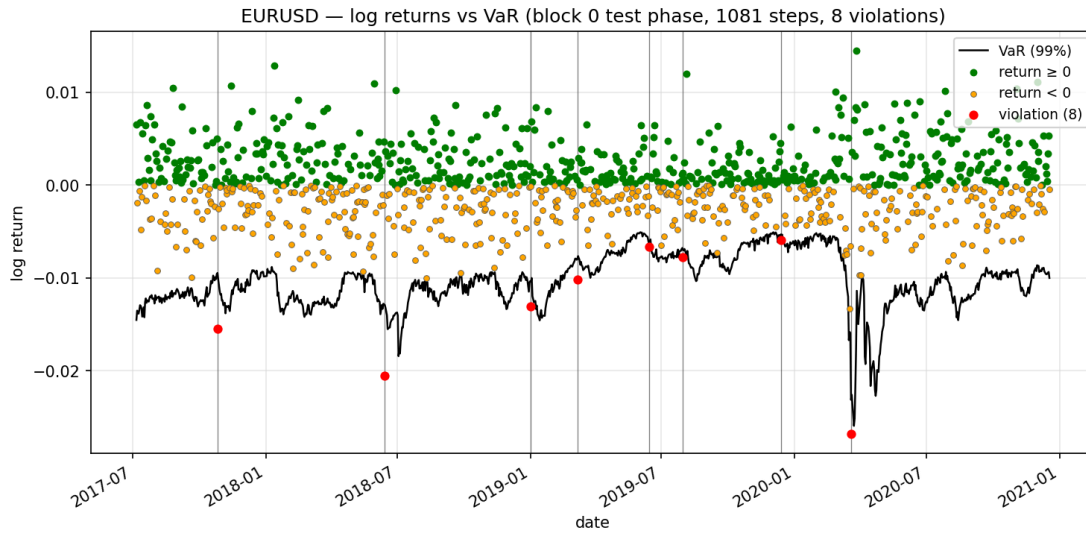


Figure 10. EUR/USD test phase: realised log returns against the predicted 99 % VaR (black). The VaR line tracks the volatility regime — it deepens sharply in March 2020 and contracts again afterwards — and the eight violations are dispersed rather than clustered, which is what the independence test formalises.

9 Benchmark against DeepVaR

9.1 What DeepVaR does, and how the protocols differ

DeepVaR (Fatouros et al., *Digital Finance* 5:29–56, 2023) wraps Salinas et al.'s DeepAR — an autoregressive LSTM that outputs the parameters of a likelihood and is sampled by Monte Carlo — in a continual-retraining loop. At every step the model is refitted on the most recent 900 daily log returns (two LSTM layers, 50 cells, 15-day context, five epochs, Adam at 10^{-4} , ten random seeds bagged), 1 000 samples are drawn, and VaR is the empirical 1 % quantile. **The architectural philosophies are almost opposite:** DeepVaR is a short-memory model retrained from scratch about a thousand times, at roughly 12.5 s per estimate; BCPNN-LTSL is a single long-memory model trained once and then run frozen over the test period (but importantly, capable of continual learning, should there be more incoming data at any time). At 744 s for the entire four-pair study including training and testing, BCPNN-LTSL is much cheaper and inference steps take less than 50ms.

Table 7. Protocol comparison. For the head-to-head tables below we recomputed our metrics on DeepVaR's exact window; no re-training was needed, since the model runs frozen through the whole test phase.

	BCPNN-LTSL	DeepVaR
temporal memory	explicit LTSL, 1 – 97 day receptive fields	15-day context window, 900-day refit sample
training	one continual pass, Bayesian–Hebbian traces, no gradients	refit per step, SGD, 10-seed bagging
predictive law	non-parametric 79-bin pmf	parametric likelihood, 1 000 MC samples
cross-instrument	one shared network, sequential concatenation	joint multivariate fit, cross-learning
compute for the study	744 s total (single GPU run), <50ms per step.	$\approx 12.5 \text{ s} \times 928 \text{ steps per asset}$
backtest window	2017-07-06 – 2020-12-18 (1 081) or matched 929	2018-01-01 – 2020-12-18 (928)

9.2 Two problems in the published loss definitions

DeepVaR's evaluation suite is adopted here for comparability, but two of the four definitions as printed do not behave as described.

Tick loss (their Eq. 13). The published expression is $(\alpha - I_t)(\text{PnL}_t - \text{VaR}_t)$, which for $\alpha = 0.99$ weights the ninety-nine-times-more-numerous non-exceedances by 0.99 and the rare breaches by -0.01 — the inverse of the pinball polarity for a 1 % quantile. Evaluated as printed it would return values of order 8×10^{-3} ; the paper reports values of order 2×10^{-4} , which is exactly what the corrected form $((1-\alpha) - I_t)(\text{PnL}_t - \text{VaR}_t)$ yields. We therefore conclude that the **typographical error is in the manuscript and not in their code**, and that their published tick losses are directly comparable with ours. We use the corrected form throughout.

Smooth loss (their Eq. 12). With $m = \text{PnL} - \text{VaR}$ as printed, the loss is positive and penalises excess conservatism, which is the intended behaviour. The published values are negative, which is reproduced only by taking $m = \text{VaR} - \text{PnL}$; under that convention the score decreases monotonically as VaR is made deeper, so it rewards unbounded conservatism and cannot rank models. Our values are positive and of the same magnitude, but we regard this metric as **not directly comparable** and exclude it from the head-to-head verdict.

Quadratic loss is coverage in disguise. $l_{\text{QL}} = \text{mean } l_{\text{t}}(1 + (\text{PnL}-\text{VaR})^2)$ and the squared term is of order 10^{-4} , so the metric equals the violation rate to five decimal places — in their tables l_{QL} and r_v are literally identical in every row, and the same holds for ours. It is thus a pure under-coverage score, minimised by reporting an arbitrarily deep VaR, and it says nothing about whether a model is *calibrated*. We report it for completeness but weight it accordingly.

9.3 Head-to-head on the identical window

Restricting our frozen model's output to 2018-01-01 – 2020-12-18 gives 929 scored steps against DeepVaR's 928 (a one-day boundary difference; expected violations 9.29 versus 9.28). No refitting is involved: the model was trained on data ending 2017-06-11 and simply runs forward.

Table 8. Matched-window comparison, 2018-01-01 to 2020-12-18, $\alpha = 0.99$. DeepVaR figures are as published (Tables 2–9 of Fatouros et al.); marks rejection at 5 %, * at 1 %. Our JPY series is JPYUSD where theirs is USDJPY, so that row compares opposite sides of the same market.

pair	model	viol.	rate	quad.	tick $\times 10^{-4}$	firm	LR _{uc}	LR _{ind}	LR _{cc}
AUDUSD	BCPNN–LTSL	11	1.18 %	0.01184	1.93	0.01310	0.30	0.26	0.56
AUDUSD	DeepVaR	5	0.54 %	0.00539	2.00	0.02334	2.39	0.05	2.44
GBPUSD	BCPNN–LTSL	13	1.40 %	0.01399	1.87	0.01358	1.33	1.88	3.21
GBPUSD	DeepVaR	7	0.75 %	0.00754	1.90	0.02454	0.61	4.26*	4.87
JPYUSD	BCPNN–LTSL	12	1.29 %	0.01292	1.48	0.01046	0.73	2.16	2.89
JPYUSD	DeepVaR	8	0.86 %	0.00862	1.50	0.02146	0.18	0.14	0.32
EURUSD	BCPNN–LTSL	7	0.75 %	0.00754	1.21	0.01004	0.62	0.11	0.73
EURUSD	DeepVaR	2	0.22 %	0.00216	1.40	0.01501	8.46**	0.01	8.47*
mean	BCPNN–LTSL	10.8	1.16 %	0.01157	1.62	0.01179	0.75	1.10	1.85
mean	DeepVaR	5.5	0.59 %	0.00593	1.70	0.02109	2.91	1.11	4.02

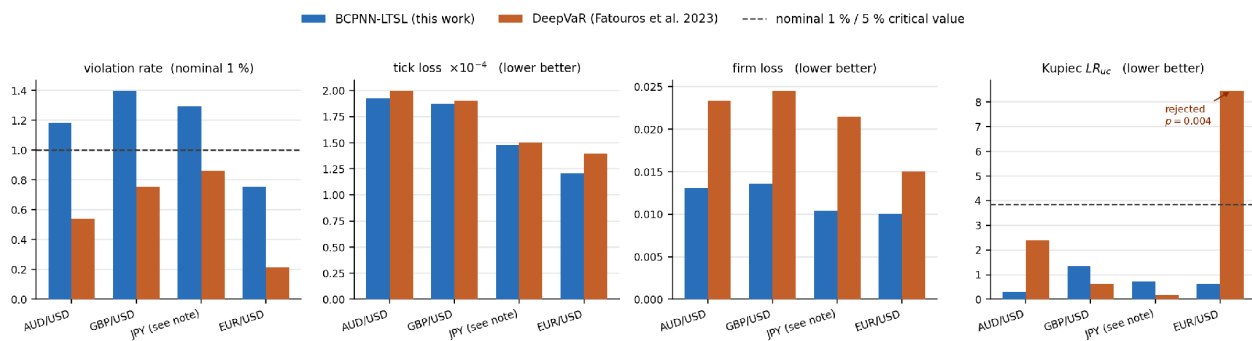


Figure 11. Matched-window comparison. Violation rate: DeepVaR sits well below the nominal 1 % on all four pairs and is rejected on EUR/USD; ours brackets the target. Tick loss: effectively a tie except on EUR/USD. Firm loss: a decisive advantage for BCPNN–LTSL, because its VaR is far less deep. Kupiec statistic: lower is better calibrated.

9.4 Reading the comparison

Where BCPNN–LTSL wins.

- **Coverage.** All twelve of our tests pass (LR_{uc} 0.30–1.33, LR_{ind} 0.11–2.16, LR_{cc} 0.56–3.21). DeepVaR is rejected twice: EUR/USD unconditional coverage at the 1 % level (LR_{uc} = 8.46, $p = 0.004$) and, jointly, conditional coverage (LR_{cc} = 8.47, $p = 0.014$); and GBP/USD independence at 5 % (LR_{ind} = 4.26, $p = 0.039$). Their mean Kupiec statistic is 2.91 against our 0.75.

Why the DeepVaR coverage rejections matter:

1. *EUR/USD fails coverage.* Two breaches where 9.28 were expected — rejected at the 1% level, when the true confidence level is close to 99.8%. Their mean VaR of roughly -0.021 (readable off their firm loss) is nearly twice as deep as ours. Theoretically, a bank running it this holds almost twice the capital required by a 99% level, so can't meaningfully compare that number against our 99% VaR.
2. *GBP/USD fails independence.* Their seven breaches are not scattered — conditional on a breach, the next day is disproportionately likely to breach too, suggesting their model has trouble to catch up. No model is perfect of course, but their paper explicitly claims the opposite: "DeepVaR is the only model that avoids clustered VaR violations due to its continual learning." Their own Table 5 contradicts that for GBP/USD, and their Dynamic Quantile test agrees ($DQ = 27.8$, $p = 0.000$), so it is arguably two failures on that pair alone.

- **Capital efficiency.** Our Firm loss is 0.0118 against their 0.0211, a 44 % reduction, reflecting a mean VaR of -0.0119 against roughly -0.0211 . DeepVaR achieves its low violation count by holding almost twice the capital.
- **Computational Cost.** One single run of 744 s for training and testing off all four currency pairs versus a per-step refit at ≈ 12.5 s. Once trained, our marginal cost per new day is a single forward pass, less than 50ms. A smaller contribution to this efficiency is also due to the logarithmic spacing of LTSL representation. A linear spacing of the

same total number of time constants would dedicate almost the entire computational budget to redundant, very slow filters while severely under-sampling the rapid, short-term dynamics relevant to the next-day prediction. Geometric spacing (constant consecutive ratio) is what preserves scale covariance across the network's multi-scale memory and arguably improves computational efficiency.

- **Continual Learning.** Because our learning time constant defaults to the length of the training stream ($\tau_p = \text{trnpat} \cdot \Delta t$) and the probabilistic floor defaults to $\varepsilon = 1/(1+\text{trnpat})$, the network accumulates its Bayesian statistics over an exponential window as long as the training set itself. So the feed forward learning and testing schedule implements a genuine continual learner rather than a sequence of independent fits. Because it remains feed-forward and doesn't have independence violations like DeepVaR, it's arguably a better continual learning system than DeepVaR, even if the comparison on four forex datasets is not very comprehensive.

Where DeepVaR wins, or where the comparison is a wash.

- **Quadratic loss** favours DeepVaR by roughly a factor of two — but, as Section 9.2 shows, that metric is the violation rate, so it rewards exactly the under-coverage that the Kupiec test rejects. We do not regard it as evidence either way.
- **Tick loss** is essentially a tie. At the two significant digits the paper reports, we are ahead on AUD (1.93 versus 2.00×10^{-4}) and EUR (1.21 versus 1.40×10^{-4}) and level on GBP and JPY. The 14 % EUR/USD margin is the only individually convincing difference. Note that the full 1 081-day window gives us better-looking tick losses ($1.26\text{--}1.83 \times 10^{-4}$) purely because it includes the calm second half of 2017; the matched-window numbers are the fair ones.
- **Point forecasting.** DeepAR is a genuine conditional-mean model. Ours is not (Section 8.1). If the application needs a directional signal, this architecture does not currently supply one, but initial testing suggests that a BCPNN-LTSL with a directional signal would be inferior to DeepAR.
- **Portfolio VaR.** DeepVaR aggregates asset-level VaR to portfolio VaR through a variance–covariance matrix and evaluates 1 000 random portfolios. We evaluate asset-level VaR only; the cross-sectional dependence structure is not modelled.

The overall reading is that the two models make opposite errors. DeepVaR is systematically conservative — its pooled violation rate is 0.59 % against a 1 % target — which buys it a low violation count and a low quadratic loss at the price of a rejected coverage test and nearly double the regulatory reserve capital. BCPNN-LTSL is calibrated, pooling at 1.16 % on the matched time window and 1.04 % on the full one, and converts that calibration directly into a much cheaper risk charge. For a firm choosing between them, the relevant question is whether it is being scored on breach count or on the cost of the capital it must hold.

10 Limitations and threats to validity

- **One block, one seed.** The FOREX calendar supports exactly one multislice block, so the continual-learning claim is exercised across instruments but not across time on this dataset; the six-block evidence comes from S&P 100. The model is deterministic given its parameter file, so there is no seed variance to report — but equally there is no ensemble and no confidence interval on the point estimates beyond the coverage tests.
- **The JPY row compares inverted pairs.** Our dataset carries JPYUSD, DeepVaR's carries the inverted USDJPY, so the two studies price opposite tails of the same market. The distributions are close to symmetric, but the row is not strictly comparable, we will fix this minor issue on the next code release though.
- **The VaR floor is a real constraint.** The reportable quantile can never be deeper than the leftmost bin centre. It is comfortable here (-0.0264 against a mean VaR of -0.0119) but any application at a more extreme level, or on a heavier-tailed asset, must re-check it.
- **No expected shortfall, no portfolio aggregation, no DQ test.** We have yet to run this framework for portfolio analysis. We have not evaluated expected shortfall and do not report the Dynamic Quantile test that DeepVaR uses alongside Christoffersen's.
- **Transaction-level realism.** All results are on daily closes with no bid-ask spread, no rollover and no execution model, and the model has no directional edge, so nothing here constitutes a trading strategy.

11 Conclusions and further work

A Bayesian Confidence Propagation Network reading a logarithmic temporal scale ladder produces well-calibrated one-day-ahead predictive distributions for daily foreign-exchange returns. On the DeepVaR dataset and backtest window it passes every unconditional-coverage, independence and conditional-coverage test on all four pairs, where the published DeepVaR results are rejected on two, and it does so while reporting a 99 % VaR that is 44 % less capital-intensive at equal-or-better tick loss. The architecture achieves this without gradients, without epochs, and with a single 744-second training run that is then frozen for a three-year out-of-sample period.

As a group who has not been working a lot with financial data models before, the route there was as informative as the destination. Replacing a trailing-mean target with a signed-return target and a volatility side channel turned a

model that never beat persistence into one that always does, and the transition analysis that motivated the change (volatility autocorrelation 0.27, sign persistence 0.48) also predicts the result of Section 8.1: there is no directional edge to be had (finance is hard), so a distributional model is not a compromise but the correct formulation. Getting the tail right then required three unglamorous fixes — a scale-invariant kernel width, actually forwarding a good regularisation floor, and analytically removing that floor from the learned prior — each of which was invisible under an RMSE objective but decisive under a 99 % quantile objective.

The most promising directions are:

- **Proper out-of-sample hyperparameter selection**, by nesting the evolutionary search inside earlier calendar blocks on a longer dataset so that the tail objective can be scored on genuinely held-out validation data.
- **Expected shortfall and multi-horizon risk**, both of which are direct read-outs of the existing pmf and require no architectural change.
- **Portfolio-level aggregation**, which needs a joint code across instruments rather than the sequential concatenation used here — the natural BCPNN formulation is additional hypercolumns rather than a covariance matrix.
- **The compute-efficiency hypothesis**, which motivated the LTSL in the first place: that a logarithmic ladder represents the relevant past more cheaply than a fixed context window. The 47.6-hour search fixed $K = 15$ temporal stages against DeepVaR's 15-day context and 900-day refit sample, but a fully controlled comparison of representational cost against forecast quality has not yet been run.
- **Stress-period conditioning**, in particular whether the ladder's slow temporal stages can be shown to carry the regime information that makes the March 2020 expansion of the credible bands happen early enough.

References

- Lansner, A. and Ekeberg, Ö. (1989). A one-layer feedback artificial neural network with a Bayesian learning rule. *International Journal of Neural Systems* 1(1), 77–87.
- Lansner, A. and Holst, A. (1996). A higher order Bayesian neural network with spiking units. *International Journal of Neural Systems* 7(2), 115–128.
- Sandberg, A., Lansner, A., Petersson, K. M. and Ekeberg, Ö. (2002). A Bayesian attractor network with incremental learning. *Network: Computation in Neural Systems* 13(2), 179–194.
- Tully, P. J., Hennig, M. H. and Lansner, A. (2014). Synaptic and nonsynaptic plasticity approximating probabilistic inference. *Frontiers in Synaptic Neuroscience* 6, 8.
- Lindeberg, T. (2023). A time-causal and time-recursive scale-covariant scale-space representation of temporal signals and past time. *Biological Cybernetics* 117, 21–59.
- Lindeberg, T. and Fagerström, D. (1996). Scale-space with causal time direction. *ECCV 1996*, LNCS 1064, 229–240.
- Fatouros, G., Makridis, G., Kotios, D., Soldatos, J., Filippakis, M. and Kyriazis, D. (2023). DeepVaR: a framework for portfolio risk assessment leveraging probabilistic deep neural networks. *Digital Finance* 5, 29–56.
- Salinas, D., Flunkert, V., Gasthaus, J. and Januschowski, T. (2020). DeepAR: probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting* 36(3), 1181–1191.
- Kupiec, P. H. (1995). Techniques for verifying the accuracy of risk measurement models. *Journal of Derivatives* 3(2), 73–84.
- Christoffersen, P. F. (1998). Evaluating interval forecasts. *International Economic Review* 39(4), 841–862.
- Sarma, M., Thomas, S. and Shah, A. (2003). Selection of Value-at-Risk models. *Journal of Forecasting* 22(4), 337–358.
- González-Rivera, G., Lee, T.-H. and Mishra, S. (2004). Forecasting volatility: a reality check based on option pricing, utility function, value-at-risk, and predictive likelihood. *International Journal of Forecasting* 20(4), 629–645.
- Mackey, M. C. and Glass, L. (1977). Oscillation and chaos in physiological control systems. *Science* 197(4300), 287–289.